



School of Computer Science

COMP 8347 “Internet Applications and Distributed
Systems”

Summer 2026

Week 6

CMS Concepts and Frameworks; Web Security Case Study

DR. OLENA SYROTKINA



University of Windsor

Copyright Notice

This content is protected and may not be shared, uploaded, or distributed. Any unauthorized distribution, reproduction or sharing of this material, including uploading to **CourseHero**, **Chegg**, **StuDocu** or other websites, is **strictly prohibited** and may be subject to legal action. Students are granted access to this material solely for their personal use and may not use it for any other purpose without the express written consent of the instructor. Thank you for respecting our intellectual property rights.

Agenda

- What is a CMS (Content Management System)?
- Comparing CMS frameworks: WordPress, Drupal, Django CMS
- Inside Django CMS – how it extends what we have learned
- Web security in CMSs – common attacks (SQL injection, CSRF, XSS)
- Security case study: “The SQL Injection That Exposed a University’s Database”
- Best practices: How Django and modern CMS platforms defend against these risks

Introduction: What Is a CMS?

A **CMS (Content Management System)** is software that:

- ✓ Lets users create, edit, and organize content (text, images, videos).
- ✓ Stores that content in a database.
- ✓ Displays it using templates.
- ✓ Provides user roles and permissions

A CMS is like a library system:

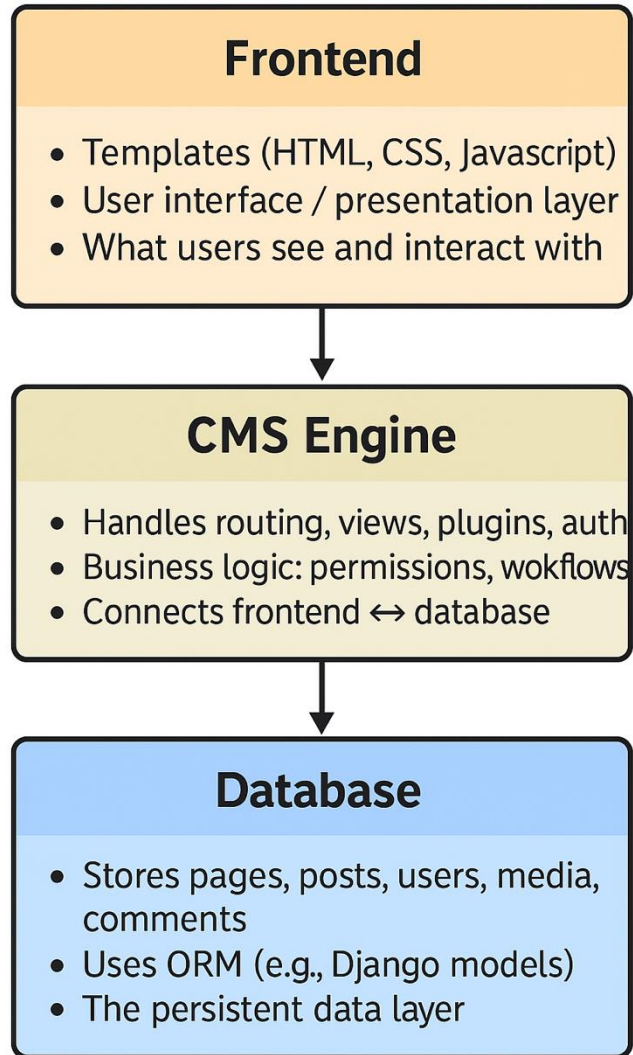
- ✓ The database is the shelves of books.
- ✓ The templates are how books are displayed.
- ✓ The admin interface is the librarian's desk.



Common CMS Examples

CMS	Language	Key Features	Used For
WordPress	PHP	Easiest for blogs, themes, plugins	Blogs, news, small business sites
Drupal	PHP	Highly customizable, modular	Government or enterprise sites
Django CMS	Python	Built on Django's MVT	Educational and corporate content sites
Ghost	Node.js	Clean, minimal, focused on writing	Modern publishing platforms

CMS Architecture: The Three Layers of Content Management



This diagram illustrates how a **Content Management System (CMS)** is structured in three interconnected tiers:

1. Frontend (Presentation Layer):

The user-facing part – templates built with HTML, CSS, and JavaScript that display pages, images, and posts to visitors.

2. CMS Engine (Logic Layer):

The “brain” of the system – handles routing, views, authentication, plugins, permissions, and workflow logic. It decides what content to serve and how to present it.

3. Database (Data Layer):

The foundation – stores persistent data like pages, posts, users, media files, and comments. Django’s ORM communicates with this layer to retrieve or save data.

Data Flow:

Requests move downward from the frontend to the database, and responses (content) flow back up through the CMS engine to the user’s browser.

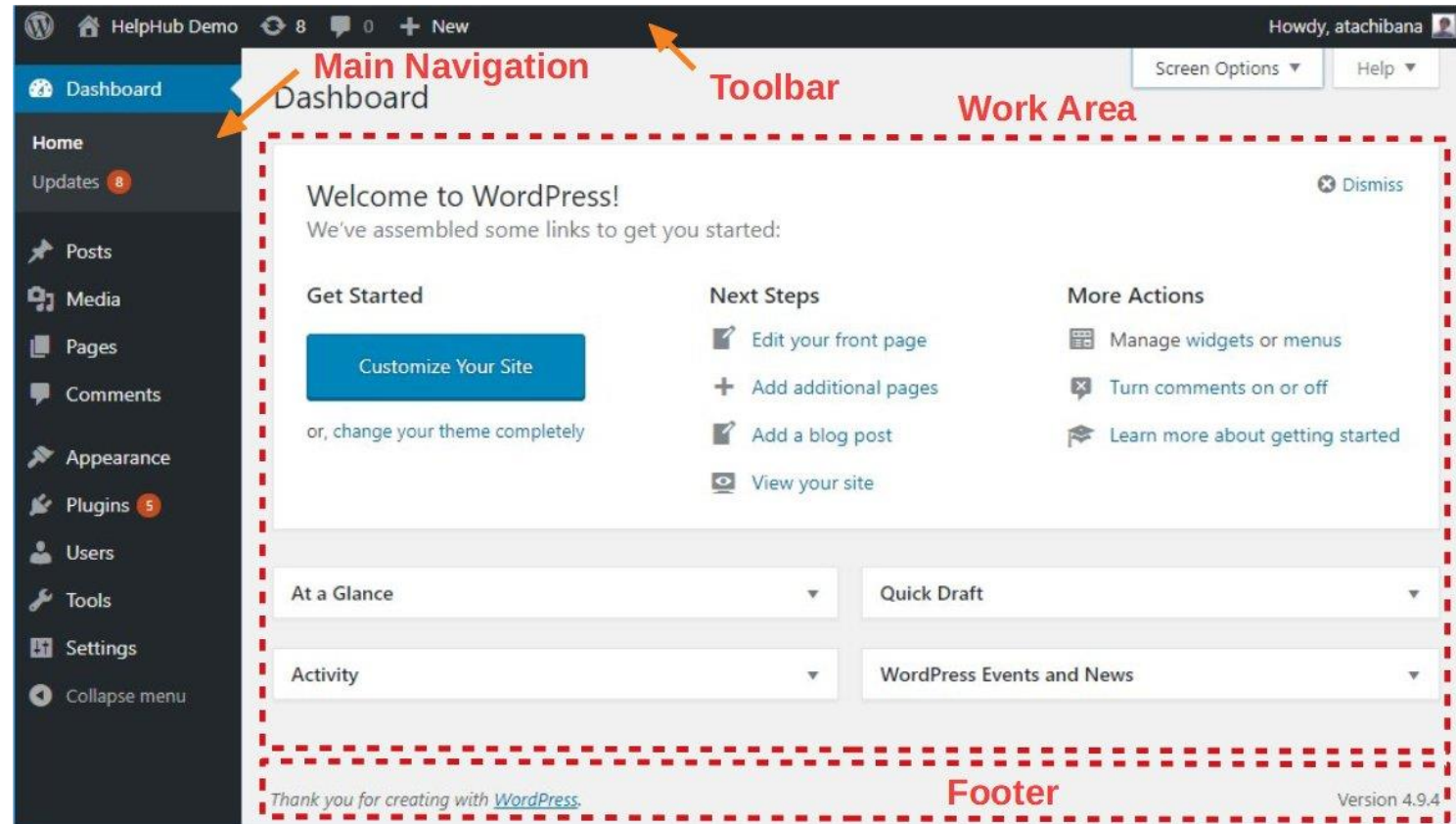
Comparing CMS Architectures

1. WordPress (most popular)

WordPress is the world's most widely used Content Management System (CMS).

It was first released in 2003 as a simple blogging platform and has since evolved into a full-featured system for building almost any kind of website – from personal blogs to e-commerce stores and corporate portals.

- ✓ Language: PHP
- ✓ Database: MySQL
- ✓ Architecture: Follows a basic LAMP stack (Linux, Apache, MySQL, PHP).
- ✓ Market share: Powers about 40% + of all websites worldwide.



Comparing CMS Architectures

1. WordPress: key components

Component	Description
Core	The foundation that provides built-in features such as posts, pages, media uploads, and user management.
Themes	Control the look and layout (e.g. “skins” for your site).
Plugins	Extend functionality – for SEO, contact forms, security, etc.
Widgets	Small reusable UI elements (e.g. sidebars or menus).
Admin Dashboard	The interface for creating and managing content, users, and plugins.

Comparing CMS Architectures

1. WordPress Content Types

WordPress organizes information using two main built-in types:

1.Posts → Dynamic, time-based entries (e.g., blog updates or news).

2.Pages → Static content (e.g., “About Us”, “Contact”, “Privacy Policy”).

Additional content types (like “Products” or “Events”) can be added via plugins or custom development.

Extensibility

One of WordPress’s greatest strengths is its **plugin ecosystem**. There are over **60,000+ plugins** available in the WordPress marketplace – many free, some paid.

Examples:

Yoast SEO – improves search visibility

WooCommerce – adds e-commerce features

Elementor – drag-and-drop website builder

Jetpack – adds security and analytics tools

However, with great flexibility comes great responsibility.

Comparing CMS Architectures

1. WordPress

Security & Maintenance Concerns

While WordPress is easy to extend, too many plugins or poorly coded ones can cause:

- ✓ Site slowdowns
- ✓ Conflicts between plugins
- ✓ Security vulnerabilities (hackers target outdated plugins)
- ✓ Version incompatibility after updates

This means admin users must be cautious about what they install.

Example:

A university news site might use:

- ✓ WordPress Core for posts and pages
- ✓ Custom theme with university branding
- ✓ Events plugin for academic calendar
- ✓ Security plugin (e.g. Wordfence)
- ✓ SEO plugin for visibility

Comparing CMS Architectures

1. WordPress

Discussion Questions:

- ☐ Why do non-technical users prefer WordPress?
- ☐ What problems might occur when too many plugins are installed?
- ☐ How is Django different in terms of plugin management?
- ☐ Which kind of project would not be a good fit for WordPress?

Comparing CMS Architectures

1. WordPress

Disadvantages of WordPress

Challenge	Explanation
Security Risks	WordPress is a popular attack target; many hacks come from outdated plugins or themes.
Performance	Too many plugins and database queries slow down large sites.
Maintenance	Frequent updates for core, plugins, and themes are required to stay secure.
Limited Customization (Backend)	Developers are constrained by PHP structure and plugin availability.
Plugin Conflicts	Third-party extensions can break functionality.
Database Bloat	Over time, revisions, logs, and plugin data clutter the database.
Not Ideal for Complex Logic	Lacks strong support for structured business workflows or analytics.

Comparing CMS Architectures

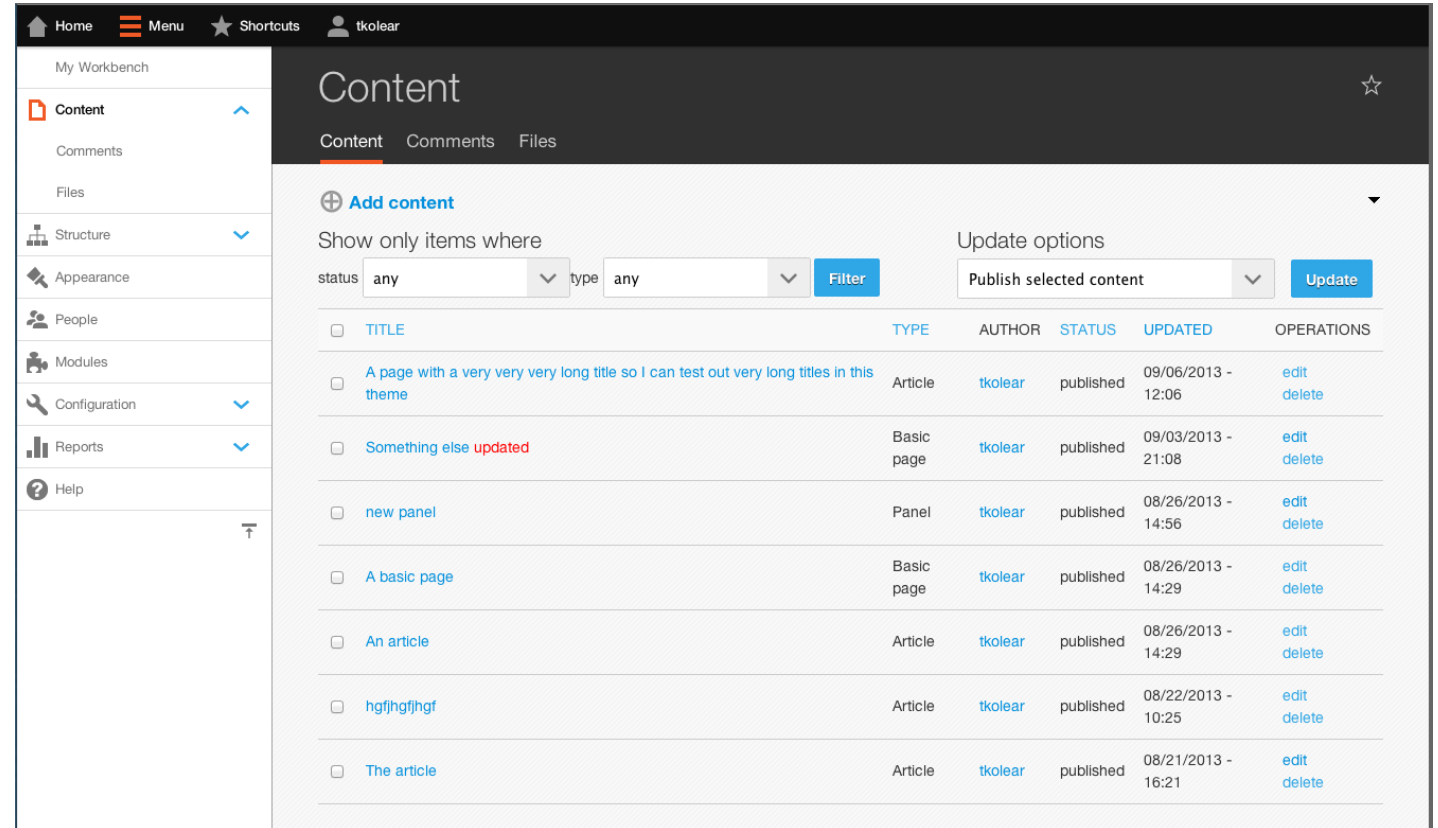
2. Drupal

Drupal is a free and open-source CMS written in PHP. It uses a modular architecture – meaning every piece of functionality (articles, users, menus, comments) is powered by a module.

It stores data in a relational database (MySQL or PostgreSQL) and separates:

- ✓ Frontend (themes) → how the website looks
- ✓ Backend (core + modules) → how it works
- ✓ Database → where the content lives

Drupal's design is based on entities and fields. An entity is any type of data object (e.g., article, user, comment). Each entity has fields (like title, body, date, or tags). Developers and admins can easily create new entities or fields, giving Drupal a flexibility similar to Django's models.



Comparing CMS Architectures

2. Drupal: key components

Feature	Description
Language	PHP
Database	MySQL or PostgreSQL
Architecture	Modular, extensible, built around nodes and entities
Used by	Government of Canada, NASA, Oxford University, UNICEF
Goal	Enterprise-grade content management with deep customization and security

Comparing CMS Architectures

2. Drupal: how does it work internally?

Frontend (Themes)

HTML, CSS, Twig Templates, Blocks

Drupal Core

Nodes, Entities, Views, Routing,
Permissions, Field API, Caching
Modules

Database

Stores content, users, taxonomy,
configuration

This diagram illustrates the three main layers of Drupal's system architecture:

1. Frontend (Themes):

This is the user-facing layer where Drupal renders web pages using **HTML, CSS, Twig templates, and Blocks**. It controls how content is displayed to visitors through themes and layout components.

2. Drupal Core:

The central engine that manages all content and logic. It handles **Nodes, Entities, Views, Routing, Permissions, and the Field API**. The core also controls caching, module interactions, and system-level configurations.

3. Database:

The foundation where Drupal stores all data – including **content, users, taxonomy (tags and categories), and site configuration**. The database provides structured and relational storage for the entire site.

Comparing CMS Architectures

2. Drupal

Modules – The Building Blocks of Drupal

Core Modules (built-in)

- **Node** – handles content
- **User** – handles authentication
- **Taxonomy** – manages tags and categories
- **Views** – allows creating filtered lists of content (comparable to a “query builder”)
- **Field UI** – adds fields to entities
- **Menu** – navigation system

Contributed Modules (installable)

- **Pathauto** – automatically generates SEO-friendly URLs
 - **Webform** – creates complex forms
 - **Views Bulk Operations** – mass-editing of content
 - **Metatag** – adds SEO metadata
 - **Paragraphs** – flexible page layout builder
- Unlike WordPress plugins, Drupal modules integrate more deeply and rarely conflict because they follow strict architectural conventions.*

Comparing CMS Architectures

2. Drupal

Security & Maintenance

Drupal is known for excellent security – one reason governments and universities use it.

- ✓ Regular security bulletins and patches
- ✓ Built-in form tokens (similar to Django's CSRF)
- ✓ Access control at field, view, and content level
- ✓ Fine-grained permission management
- ✓ Centralized configuration management for safe deployment

Comparing CMS Architectures

2. Drupal

Discussion Questions:

- ☐ Why do governments choose Drupal?
- ☐ What makes Drupal harder to learn than WordPress?
- ☐ Can Drupal handle structured data like Django?
- ☐ When would you prefer Drupal over Django?

Comparing CMS Architectures

2. Drupal

Disadvantages of Drupal

Disadvantage	Explanation
Steeper learning curve	Complex admin UI and terminology (nodes, taxonomy, blocks, etc.)
Requires developer setup	Non-technical users may struggle without training
Heavy system	More database tables, higher memory usage
Slower to build small sites	Overkill for a personal blog or small business site
Module compatibility	Some contributed modules lag behind core updates
Harder theming	The Twig templating system is powerful but technical

Comparing CMS Architectures

2. Drupal

Comparison with WordPress & Django

Feature	WordPress	Drupal	Django
Language	PHP	PHP	Python
Audience	Bloggers, small business	Large orgs, enterprise	Developers, data-driven apps
Custom data types	Custom Post Types	Entities & Fields	Models
ORM	 No	 Partial (Field API)	 Full
Security	Medium	Excellent	Excellent
Ease of use	Easy	Difficult	Developer-oriented
Plugins/Modules	Very many	Stable and structured	Apps (fully controlled)

Comparing CMS Architectures

3. Django CMS – The Developer’s CMS

What Is Django CMS?

Django CMS is an open-source content management system built on top of Django, the Python web framework.

It was designed for developers who need to build secure, scalable, and flexible websites but also want to give non-technical users (like editors or managers) a simple way to edit content.

In other words, it merges two worlds:

- ✓ Django framework – for developers who write logic, define models, and control structure.
- ✓ CMS interface – for editors who drag, drop, and edit pages easily.

Comparing CMS Architectures

3. Django CMS – Core Technologies

Component	Technology
Language	Python
Framework	Django
Database	Any Django-supported DB (SQLite, PostgreSQL, MySQL)
Architecture	MVT (Model–View–Template)
Template Engine	Django templates
Authentication	Uses Django’s built-in auth system
ORM	Django ORM – same as in your apps

Comparing CMS Architectures

3. Django CMS – Architecture Diagram

Frontend (Templatas)

HTML, CSS, and user-editable page regions

Django CMS Core

Page structure, plugins, permissions, toolbar
Built-in admin interface for content editing

Django Framework

ORM, Authentication, Middleware, Routing

Database Layer

Content models, users, custom data

This diagram shows how Django CMS is layered on top of the Django framework to create dynamic, structured websites.

1.Frontend (Templates):

Handles how content is displayed to users through **HTML**, **CSS**, and **editable regions**. Django CMS allows content editors to modify these areas directly without coding.

2.Django CMS Core:

The content management layer responsible for **page structure**, **plugins**, **permissions**, and the **toolbar**. It integrates deeply with Django's admin interface, letting users edit pages visually.

3.Django Framework:

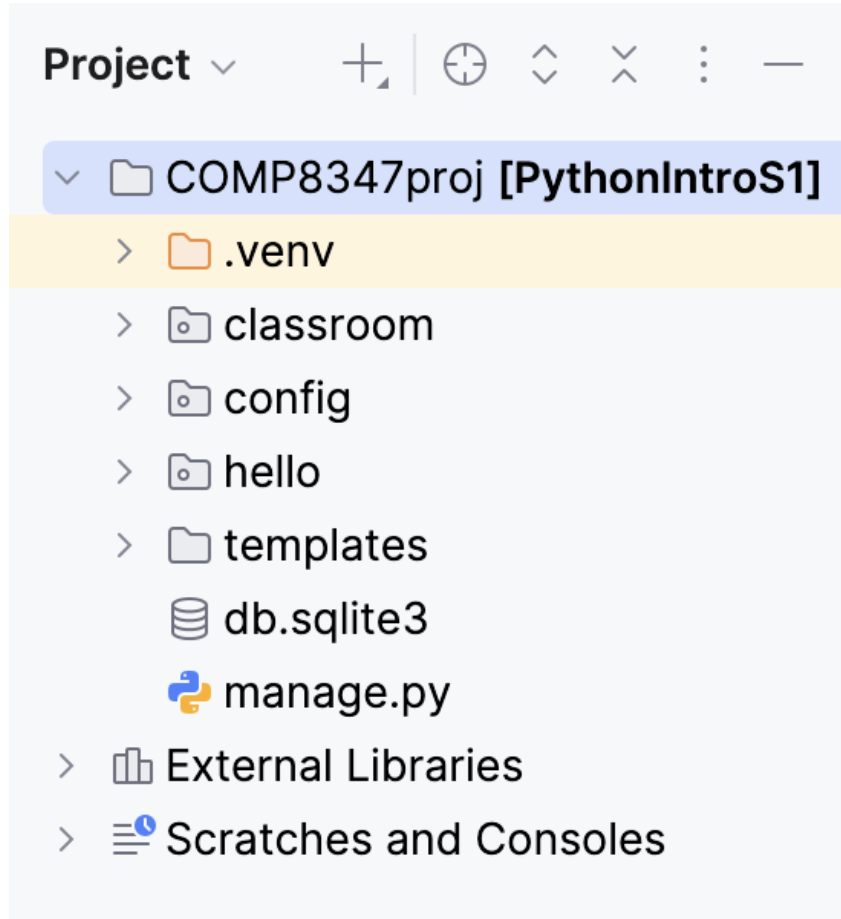
Provides the foundation – **ORM**, **authentication**, **middleware**, and **routing** – for handling logic, user sessions, and data flow between the CMS and the database.

4.Database Layer:

Stores **content models**, **user accounts**, and **custom data** defined by developers. Django's ORM keeps the database synchronized with Python model definitions.

Comparing CMS Architectures

3. Django CMS – How It Works in Practice?



When you create a page in Django CMS:

- ✓ The page structure is stored in the database (through Django ORM).
- ✓ Templates define editable “content placeholders.”
- ✓ Editors can log in, drag content blocks into placeholders, and save changes.
- ✓ Django’s MVT pattern handles rendering, just like in a normal Django app.

Comparing CMS Architectures

3. Django CMS – Security and Integration

Because it's built on Django, Django CMS inherits all of Django's strengths:

- ✓ CSRF protection
- ✓ XSS prevention (auto-escaping templates)
- ✓ Secure user authentication
- ✓ Robust ORM that prevents SQL injection
- ✓ Permissions and groups
- ✓ Reusable apps (you can connect your own models easily)

Example:

You can link your existing Django Student model to CMS pages and display student profiles dynamically using Django's ORM – no need for a separate plugin.

Comparing CMS Architectures

3. Django CMS – Advantages

Advantage	Description
Secure by design	Built on Django's hardened security model
Integrates with Django apps	Reuse any Django app or model
Developer control	You decide what models, templates, and logic exist
Scalable	Supports enterprise-level projects
Multi-language support	Built-in i18n features
Open source & well-documented	Large developer community

Comparing CMS Architectures

3. Django CMS – Disadvantages

Disadvantage	Explanation
Developer-dependent setup	Needs Django knowledge – not for complete beginners
Smaller plugin ecosystem	Fewer prebuilt extensions compared to WordPress
Limited drag-and-drop	Less visual design flexibility than WordPress builders
Learning curve for editors	The interface is simpler but still technical
Performance tuning	Requires configuration for caching and speed

Comparing CMS Architectures

3. Django CMS – When to Use Django CMS

-  Best choice for:
 - ✓ Universities or research portals needing both structure and flexibility
 - ✓ Secure internal tools with editable content
 - ✓ Corporate intranets with roles and permissions
 - ✓ Websites that already use Django and need CMS-like editing

-  Not ideal for:
 - ✓ Simple personal blogs
 - ✓ Teams without Python/Django skills
 - ✓ Projects requiring hundreds of prebuilt plugins

Comparing CMS Architectures

WordPress vs. Drupal vs. Django CMS

Feature	WordPress	Drupal	Django CMS
Language	PHP	PHP	Python
Framework	Custom	Modular Core	Django
Architecture	LAMP	Modular Entities	MVT
Security	Medium	High	Very High
Flexibility	Easy, limited	Complex, high	Developer-controlled
Extensibility	Plugins	Modules	Django Apps
Ease of setup	Easy	Moderate	Moderate–Advanced
Best for	Blogs, small business	Governments, large orgs	Secure, custom web apps
Database Control	Limited	Structured	Full ORM support

Comparing CMS Architectures

3. Django CMS

Discussion Questions:

- ☐ Why might a developer choose Django CMS instead of WordPress or Drupal?
- ☐ How does Django CMS combine MVT with user-friendly content editing?
- ☐ What security benefits come automatically with Django CMS?
- ☐ In what type of organizations would Django CMS be the best fit?

Installing Django CMS (Optional)

Step 1 – Create and activate a virtual environment

Windows:

```
python -m venv venv  
venv\Scripts\activate
```

Mac/Linux:

```
python3 -m venv venv  
source venv/bin/activate
```

Installing Django CMS (Optional)

Step 2 – Install Django CMS installer

```
pip install.djangocms-installer
```

This installs the official **Django CMS Installer** (a helper tool that sets up everything for you automatically).

Step 3 – Create a new Django CMS project

```
djangocms mycmsproject
```

Then it will ask you some questions (you can press Enter to accept defaults):

- ✓ Database (SQLite by default – fine for testing)
- ✓ Django version
- ✓ Language and timezone

After a few minutes, it will generate a complete Django CMS site in the `mycmsproject/` folder.

Installing Django CMS (Optional)

Step 4 – Go into your project and start the server

```
cd mycmsproject  
python manage.py runserver
```

Then open:

<http://127.0.0.1:8000/>

You will see a default Django CMS homepage!

Step 5 – Create a superuser

To log in to the CMS toolbar:

```
python manage.py createsuperuser
```

Then visit:

<http://127.0.0.1:8000/admin/>

About Django CMS

Django CMS is:

- ✓ **Free and open source** (BSD License)
- ✓ Supported by the **Django CMS Association**
- ✓ Used by major organizations (NASA, L'Oréal, PBS, etc.)
- ✓ Built with Django, so everything you already learned applies!

Helpful Links:

- ✓ Official site: <https://www.django-cms.org>
- ✓ Docs: <https://docs.django-cms.org>
- ✓ GitHub: <https://github.com/django-cms/django-cms>

Mini-CMS Lab in Your Project

Goal

➤ **Build a tiny CMS to manage blog-style “Articles” inside your classroom app.**

You will do the following:

- ✓ Define a new model called Article
- ✓ Create templates to list and display them
- ✓ Add admin management
- ✓ (Optional) Allow adding via form

This will simulate a **content management workflow** – the same ideas that power Django CMS or WordPress, but simplified and visible.

Mini-CMS Lab in Your Project

Step 1 – Create a Model for Articles

In your classroom/models.py, add:

```
from django.db import models
from django.utils import timezone

class Article(models.Model):
    title = models.CharField(max_length=200)
    content = models.TextField()
    author = models.CharField(max_length=100,
default="Admin")
    created_at = models.DateTimeField(default=timezone.now)

    def __str__(self):
        return self.title
```

Mini-CMS Lab in Your Project

Step 2 – Apply Migrations

Run these in your terminal:

```
python manage.py makemigrations classroom
python manage.py migrate
```

This creates the **classroom_article** table in your database – just like any CMS would store pages or posts.

```
Terminal  Local x + v
os@Olana-Syrotkina COMP8347proj % python3 manage.py makemigrations classroom
Migrations for 'classroom':
  classroom/migrations/0002_article.py
  + Create model Article

os@Olana-Syrotkina COMP8347proj % python3 manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, classroom, contenttypes, sessions
Running migrations:
  Applying classroom.0002_article... OK

os@Olana-Syrotkina COMP8347proj %
```

Mini-CMS Lab in Your Project

Step 3 – Register in Admin

Open classroom/admin.py:

```
from django.contrib import admin
from .models import Article

admin.site.register(Article)
```

Now, in the Django admin, you will have a new section called “Articles.” You can add, edit, and delete content from there – no coding needed.

Mini-CMS Lab in Your Project

Step 4 – Create Views and URLs

In classroom/views.py:

```
from django.shortcuts import render, get_object_or_404
from .models import Article

def article_list(request):
    articles = Article.objects.all().order_by('-created_at')
    return render(request, "classroom/article_list.html",
{"articles": articles})

def article_detail(request, pk):
    article = get_object_or_404(Article, pk=pk)
    return render(request, "classroom/article_detail.html",
{"article": article})
```

Mini-CMS Lab in Your Project

Step 4 – Create Views and URLs

In classroom/urls.py:

```
from django.urls import path
from . import views

urlpatterns += [
    path("articles/", views.article_list, name="article_list"),
    path("articles/<int:pk>/", views.article_detail,
name="article_detail"),
]
```

This mimics a CMS routing structure – /articles/ for the list, and /articles/1/ for individual pages.

Mini-CMS Lab in Your Project

Step 5 – Create Templates

templates/classroom/article_list.html

```
{% extends "classroom/base.html" %}
{% block content %}
    <h1>Articles</h1>
    <div class="article-list">
        {% for article in articles %}
            <div class="article-card">
                <h2><a href="{% url 'article_detail' article.pk %}">{{ article.title }}</a></h2>
                <p><small>by {{ article.author }} | {{ article.created_at|date:"F j, Y" }}</small></p>
                <p>{{ article.content|truncatewords:20 }}</p>
            </div>
        {% empty %}
            <p>No articles have been added yet.</p>
        {% endfor %}
    </div>
{% endblock %}
```


Mini-CMS Lab in Your Project

Step 5 – Create Templates

templates/classroom/article_detail.html

```
{% extends "classroom/base.html" %}
{% block content %}
    <article>
        <h1>{{ article.title }}</h1>
        <p><small>by {{ article.author }} | {{ article.created_at|date:"F j, Y, g:i a"
    }}</small></p>
        <hr>
        <p>{{ article.content }}</p>
    </article>
{% endblock %}
```

Mini-CMS Lab in Your Project

Step 6 – Add CSS (optional but recommended)

You already have styling from previous labs. Add this snippet to your main CSS file:

```
.article-card {  
    background: #f7f7ff;  
    border: 1px solid #ccc;  
    padding: 20px;  
    margin-bottom: 15px;  
    border-radius: 8px;  
}  
.article-card h2 a {  
    color: #003366;  
    text-decoration: none;  
}  
.article-card h2 a:hover {  
    text-decoration: underline;  
}
```

Mini-CMS Lab in Your Project

Step 7 – Test in the Browser

Run your server:

```
python manage.py runserver
```

Then visit:

- ✓ <http://127.0.0.1:8000/admin/> → add a few articles.
- ✓ <http://127.0.0.1:8000/articles/> → see them displayed beautifully.
- ✓ Click an article → view its detail page.

Username: **SuperAdmin**

Email: **SuperAdmin@uwindsor.ca**

Password: **SuperPass123**

Congratulations! You have built your **own lightweight CMS**, using Django's MVT pattern!

Web Security Case Study

What is SQL injection?

At a high level, SQL injection happens when untrusted input (form fields, URL parameters, headers, etc.) is concatenated or interpolated directly into SQL statements and then executed by the database. An attacker crafts input that changes the structure of the SQL query – e.g., turning a WHERE clause into a tautology or adding additional SQL commands – allowing them to read, modify, or delete data, or even escalate privileges.

How attacks typically work (conceptual)

-  Find an input point (search box, login field, GET/POST parameter).
-  Inject payload that alters the SQL statement structure (for example, by adding ' OR '1'='1' in classic textbook examples).
-  Database executes the modified SQL and returns data or performs actions the developer never intended.
-  Exploit & extract: attackers may enumerate data, pivot to other systems, or escalate access.

Web Security Case Study

The SQL Injection Disaster

SQL Injection = it's when raw user input alters a database query.

```
# BAD CODE
def unsafe_login(request):
    u = request.GET['user']
    p = request.GET['pass']
    sql = f"SELECT * FROM users WHERE
username='{u}' AND password='{p}' "
```

If attacker enters:

' OR '1'='1' → bypasses login.

Web Security Case Study

Famous “SQL injection disasters” (real-world cases)

♥ Heartland Payment Systems (2007–2008)

One of the largest payment-card breaches of its time. Attackers used SQL injection against web-facing systems and then moved laterally, ultimately exposing tens of millions of card numbers and causing massive cost and remediation work for the company. This incident is often cited as a turning point in awareness about web-application risk in payment systems.

♥ TalkTalk (October 2015)

An SQL injection on vulnerable web pages allowed attackers to access about 156,959 customers’ personal data and about 15,656 bank account details/sort codes. The UK Information Commissioner’s Office fined TalkTalk for failing to implement basic security measures; the breach caused heavy customer loss and multimillion-pound business impact.

♥ Sony / LulzSec (2011)

Several Sony sites were compromised via common web vulnerabilities including SQL injection; user credentials and other consumer data were exposed, contributing to broader reputational and operational fallout. This episode (and others around that time) showed how SQLi could be used to scale attacks against consumer platforms.

Demo: SQL Injection (safe, local Django example)

Goal:

- ✚ Create a small blog app with an Article model.
- ✚ Add a deliberately vulnerable view that builds SQL by concatenating user input.
- ✚ Show the vulnerability in tests (local only).
- ✚ Replace with safe alternatives: parameterized query and Django ORM.

1) Create project & app

Run in Terminal:

```
# create a virtualenv and install Django
python -m venv venv
source venv/bin/activate
pip install django
```

```
# create project and app
django-admin startproject mycmsproject
cd mycmsproject
python manage.py startapp blog
```

```
# add "blog" to INSTALLED_APPS in
mycmsproject/settings.py
```

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'blog',
]
```


Demo: SQL Injection (safe, local Django example)

2) Model: blog/models.py

```
from django.db import models

class Article(models.Model):
    title = models.CharField(max_length=200)
    body = models.TextField()
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.title
```

Run migrations:

```
python manage.py makemigrations
python manage.py migrate
```

Demo: SQL Injection (safe, local Django example)

3) Create test data (via Django shell):

```
python manage.py shell
>>> from blog.models import Article
>>> Article.objects.create(title="First", body="Hello")
>>> Article.objects.create(title="Second", body="World")
>>> exit()
```

4) Vulnerable view (do NOT deploy)

WARNING: the following code will be insecure! It demonstrates string concatenation into SQL

Demo: SQL Injection (safe, local Django example)

4) Vulnerable view (do NOT deploy)

WARNING: the following code will be insecure! It demonstrates string concatenation into SQL

Add to blog/views.py:

```
from django.http import JsonResponse
from django.db import connection
from django.shortcuts import get_object_or_404
from .models import Article

def vulnerable_article(request):
    # WARNING: insecure - demonstrates string concatenation into SQL
    article_id = request.GET.get('id', '')
    sql = f"SELECT id, title, body FROM blog_article WHERE id = {article_id};"
    with connection.cursor() as cursor:
        cursor.execute(sql)          # vulnerable if article_id contains SQL
        rows = cursor.fetchall()
    articles = [{"id": r[0], "title": r[1], "body": r[2]} for r in rows]
    return JsonResponse({"articles": articles})
```

Demo: SQL Injection (safe, local Django example)

4) Vulnerable view (do NOT deploy)

WARNING: the following code will be insecure! It demonstrates string concatenation into SQL

Add to blog/views.py:

```
def safe_article_param(request):
    article_id = request.GET.get('id', '')
    sql = "SELECT id, title, body FROM blog_article WHERE id = %s;"
    with connection.cursor() as cursor:
        cursor.execute(sql, [article_id]) # parameterized
        rows = cursor.fetchall()
    articles = [{"id": r[0], "title": r[1], "body": r[2]} for r in rows]
    return JsonResponse({"articles": articles})

def safe_article_orm(request):
    article_id = request.GET.get('id', '')
    try:
        pk = int(article_id)
    except (ValueError, TypeError):
        return JsonResponse({"error": "invalid id"}, status=400)
    article = get_object_or_404(Article, pk=pk)
    return JsonResponse({"article": {"id": article.id, "title": article.title, "body": article.body}})
```

Demo: SQL Injection (safe, local Django example)

Then ensure your blog/urls.py matches these names:

```
# blog/urls.py
from django.urls import path
from . import views

urlpatterns = [
    path('vulnerable/', views.vulnerable_article, name='vulnerable_article'),
    path('safe_param/', views.safe_article_param, name='safe_article_param'),
    path('safe_orm/', views.safe_article_orm, name='safe_article_orm'),
]
```

And include them at the project level:

```
# mycmsproject/urls.py
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('blog/', include('blog.urls')),
]
```

What's happening: the view interpolates the **id** parameter directly into SQL. If **id** contains malicious SQL, it changes the query structure.

Demo: SQL Injection (safe, local Django example)

4) Demonstration (local only)

Start server:

```
python manage.py runserver
```

Then in your lab browser (localhost) you can request:

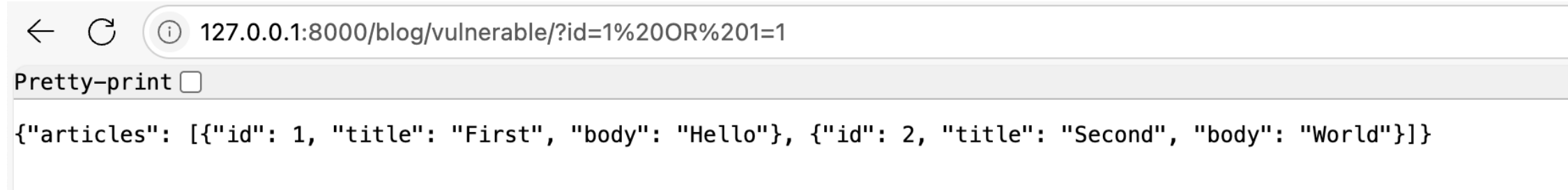
<http://127.0.0.1:8000/blog/vulnerable/?id=1> OR 1=1

http://127.0.0.1:8000/blog/safe_param/?id=1 OR 1=1

http://127.0.0.1:8000/blog/safe_orm/?id=1 OR 1=1

Demo: SQL Injection (safe, local Django example)

<http://127.0.0.1:8000/blog/vulnerable/?id=1> OR 1=1



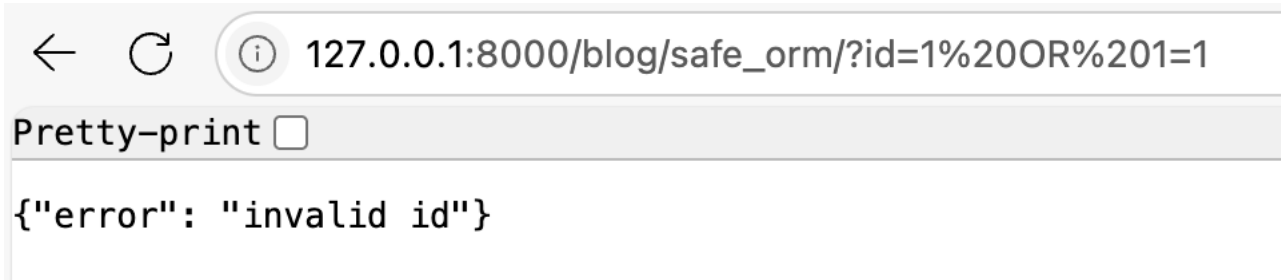
Django returned:

```
{
  "articles": [
    {"id": 1, "title": "First", "body": "Hello"},
    {"id": 2, "title": "Second", "body": "World"}
  ]
}
```

That's **both rows** in your database – even though you only asked for id=1.

Demo: SQL Injection (safe, local Django example)

http://127.0.0.1:8000/blog/safe_orm/?id=1 OR 1=1



When Django receives `id=1 OR 1=1`, it tries to convert `"1 OR 1=1"` into an integer – and fails. That triggers the `ValueError`.

This is exactly what we want – the input was rejected before any SQL query was even sent to the database.

Demo: SQL Injection (safe, local Django example)

http://127.0.0.1:8000/blog/safe_param/?id=1 OR 1=1

← ↻ ⓘ 127.0.0.1:8000/blog/safe_param/?id=1%20OR%201=1

Pretty-print ☐

```
{"articles": []}
```

Why: your **safe_param** view executes SQL with a parameter placeholder and passes the whole **id** string as a single parameter:

```
-- conceptually executed
SELECT id, title, body FROM blog_article
WHERE id = %s;
-- params: ["1 OR 1=1"]
```

So, the database looks for rows whose id equals the literal string "1 OR 1=1". No such row exists, therefore the result is an empty list. This prevents the attacker from changing the SQL logic – the input is data, not SQL.

Wrap-Up & Best Practices Summary

Risk	Prevent With
SQL Injection	ORM or authenticate()
CSRF	{% csrf_token %}
XSS	Auto-escaping ({{ }})
Weak Passwords	Hashed via pbkdf2_sha256
Data Leaks	Use HTTPS + Permissions
Plugin Vulnerabilities	Validate third-party code